

ПЛАН АВТОМАТИЗАЦИИ ТЕСТИРОВАНИЯ

American Eagle Java Automation

Объект: <https://www.ae.com/>

Параметр	Значение
Документ	Test Plan / Automation Test Plan
Проект	American Eagle java automation
Ответственный	Илья Сидорычев
Версия документа	2.0
Дата актуализации	02.10.2026
Статус	Актуальный для текущей версии проекта

1. Цели автоматизации

- Ускорить проверку критически важных пользовательских и API-сценариев.
- Снизить объём повторяющихся ручных проверок и влияние человеческого фактора.
- Раннее выявлять регрессионные дефекты после изменений в тестируемом продукте и тестовой инфраструктуре.
- Обеспечить воспроизводимый запуск API- и UI-наборов локально и в CI.
- Формировать единый Allure Report с историей запусков и публиковать его через GitHub Pages.
- Поддерживать стабильный основной CI-набор, отделяя сценарии, зависящие от ограничений production-сайта.

2. Объект и контекст тестирования

American Eagle (ae.com) — реальный production e-commerce сайт. Проект охватывает Web UI и REST API, связанные с навигацией, поиском, товарами, пользовательскими состояниями и корзиной.

- Тестирование выполняется против действующего production-окружения.
- Используются только безопасные сценарии, не приводящие к реальной покупке или оплате.
- Поведение сайта может изменяться независимо от тестового проекта, поэтому часть тестов требует адаптации к изменениям production-системы.
- Anti-bot protection может ограничивать отдельные автоматизированные пользовательские сценарии.

3. Scope — что входит в автоматизацию

3.1 UI

Область	Что проверяется	Категории
Home / Header	Доступность главной страницы, header, основных навигационных элементов и категорий.	UI, smoke / extended
Men's catalog	Переходы в разделы каталога, взаимодействие с карточками товара и гостевые/авторизованные сценарии.	UI, extended
Search	Открытие поиска и проверка актуального поведения search-сценариев.	UI, smoke / defect
Registration	Доступность регистрации, заполнение формы, клиентские проверки. Финальная отправка ограничена anti-bot.	UI, extended / positive / defect
Sign In	Позитивные и негативные проверки формы входа; часть серверных сценариев ограничена anti-bot.	UI, extended / positive / negative / defect

3.2 REST API

Область	Что проверяется	Категории
Token API	Получение guest token и проверка успешного ответа.	API
Product API	Получение информации о товарах, работа с доступными SKU, позитивные и негативные проверки.	API
Bag API	Получение корзины, добавление товара, изменение количества, удаление, работа с несколькими товарами.	API
E2E API	Полный lifecycle товара в корзине: SKU -> add -> verify -> update -> wait -> delete -> verify.	API, E2E

4. Out of Scope — что не тестируется

- Оформление реального заказа и любые операции оплаты.
- Проверка внутренних баз данных и прямой доступ к backend-хранилищам.
- Нагрузочное и стресс-тестирование production-сайта.
- Security / penetration testing.
- Деструктивные сценарии, способные повлиять на реальных пользователей или данные production.
- Стабильное подтверждение успешной регистрации/авторизации на сервере, если запрос блокируется anti-bot protection.

5. Типы, уровни и теги тестов

Тег	Назначение
UI	Web UI тесты через Selenium WebDriver
API	REST API тесты через REST Assured
smoke	Короткий набор ключевых проверок
extended	Расширенные функциональные сценарии
positive	Позитивные сценарии
negative	Негативные сценарии
E2E	Сквозной lifecycle-сценарий
defect	Сценарии с известным ограничением/изменившимся поведением; исключаются из стабильного CI-набора

6. Тестовые данные и управление состоянием

- SKU подбираются динамически через Product API, чтобы уменьшить зависимость от жёстко заданных товарных данных.
- Guest token создаётся/получается программно и используется для API-сценариев.
- Перед API-тестами состояние корзины нормализуется, чтобы тесты были независимыми друг от друга.
- Credentials не хранятся в репозитории; локально передаются через конфигурацию/системные параметры, в CI — через GitHub Secrets.
- При eventual consistency после изменения состояния корзины используется Awaitility с ограниченным временем ожидания и polling.

7. Технологический стек

Технология	Назначение
Java 21	Основной язык
JUnit 5	Тестовый фреймворк
Selenium WebDriver 4.33.0	UI-автоматизация
REST Assured	REST API тестирование
AssertJ	Fluent assertions

Технология	Назначение
Awaitility 4.3.0	Ожидание eventual consistency
Gradle 8.10	Сборка и зависимости
Owner	Конфигурация окружений
Lombok	DTO и сокращение boilerplate-кода
Allure Report	Отчётность
Git / GitHub	Контроль версий
GitHub Actions	CI
GitHub Pages	Публикация Allure Report
Selenium Standalone Chrome	Удалённый запуск UI в CI

8. Архитектура автоматизации

- Page Object Model и Component-классы для UI.
- Controller-слой для API-вызовов.
- DTO-модели для сериализации/десериализации API-данных.
- JUnit 5 Extensions для подготовки контекста и токенов.
- Steps / Support / Utils для переиспользуемой логики.
- Owner-конфигурация для работы с окружениями.
- Allure annotations для шагов и severity.
- Gradle tasks и JUnit tags для запуска отдельных наборов.

9. Окружение

Среда	Описание
Локально	Windows 10 x64; Java 21; актуальная версия Google Chrome; IntelliJ IDEA
CI	GitHub-hosted Ubuntu runner; Java 21; Selenium Standalone Chrome; Gradle Wrapper
SUT	Публичный production-сайт American Eagle и его публично доступные API, используемые проектом

10. CI/CD стратегия

```
api-tests -> api-allure-results \
            -> allure-report -> GitHub Pages
ui-tests  -> ui-allure-results /
```

- Workflow запускается при pull request в master и вручную через workflow_dispatch.
- API и UI выполняются в отдельных jobs.
- Каждый job сохраняет собственный build/allure-results как artifact.
- Отдельный allure-report job скачивает оба набора результатов, объединяет их и восстанавливает историю.
- Готовый Allure Report публикуется в gh-pages и доступен через GitHub Pages.
- Основной CI-набор исключает defect-сценарии, зависящие от известных ограничений production-сайта.

11. Allure Reporting

- Единый отчёт объединяет результаты API и UI.
- В отчёте доступны suites, test cases, severity, tags, шаги, ошибки, stack traces, attachments и история запусков.
- Публикуемая версия отчёта обновляется после успешного выполнения report job.
- Текущая ссылка на отчёт: https://ismeneger.github.io/American_Eagle_java_automation/

12. Anti-bot, ограничения и defect-стратегия

- Production-сайт использует anti-bot protection, которая может блокировать автоматизированную регистрацию и авторизацию.

- Успешная регистрация автоматизируется до отправки формы; тест завершения регистрации помечен @Disabled с явным объяснением причины.
- Сценарии с известным внешним ограничением или изменившимся поведением помечаются @Tag("defect") и не ломают основной CI.
- Negative-сценарий поиска без результатов сохранён как defect после того, как сайт начал возвращать рекомендации даже для бессмысленных запросов.
- Если production-поведение меняется, тест пересматривается: сначала проверяется, является ли изменение дефектом теста, дефектом продукта или новой ожидаемой функциональностью.

13. Риски и меры снижения

Риск	Влияние	Мера
Изменение production UI/API	Падение локаторов или контрактов	Локаторы/DTO изолированы; падение анализируется по Allure и логам
Anti-bot protection	Блокировка UI-сценариев	Disabled/defect-стратегия; отделение стабильного CI-набора
Динамические товарные данные	Невалидный SKU или изменённая доступность	Динамический выбор SKU через Product API
Eventual consistency	Состояние API обновляется не сразу	Availability с bounded polling
Сетевые/production сбой	Ложные падения	Повторный анализ, разделение API/UI jobs, сохранение artifacts
Секреты и credentials	Риск утечки	GitHub Secrets и отсутствие чувствительных данных в репозитории

14. Критерии успешного завершения

- Проект успешно компилируется (`testClasses`).
- API test suite проходит без ошибок.
- UI test suite проходит без ошибок с учётом исключённых defect/disabled сценариев.
- Критический API E2E lifecycle завершается успешно.
- API и UI формируют Allure results.
- Единый Allure Report успешно создаётся и публикуется через GitHub Pages.
- В основном CI-наборе отсутствуют необъяснённые падения.

15. Поддержка и актуализация плана

- Test Plan обновляется при существенном изменении архитектуры тестов, CI/CD, набора API/UI сценариев или внешних ограничений сайта.
- Конкретные версии браузера не фиксируются в документе, если они не являются частью тестового ограничения; используется актуальная поддерживаемая версия.
- При добавлении новых test suites рекомендуется синхронно обновлять README и данный Test Plan.

16. Ответственный

Илья Сидорычев

GitHub: <https://github.com/ISmeneger>

Repository: https://github.com/ISmeneger/American_Eagle_java_automation